

Universidad Nacional Autónoma de México



Facultad de Ingeniería

Laboratorio de Arquitectura y Organización de Computadoras

Grupo: 07 - Semestre: 2027-1

Reporte Práctica 7.

Procesador CISC 68HC11

Fecha de Entrega

13 de Octubre del 2026

Profesor

Ing. Julio Cesar Cruz Estrada

Integrantes:

Espinoza Matamoros Percival Ulises

Montiel Juárez Oscar Iván

Veraza García Amy Valentina

1. Objetivo

Diseñar un procesador CISC de 8 bits, específicamente un clon del miniprocesador 68HC11 de Motorola©, mediante una unidad de control microprogramada, e implementar nuevas instrucciones para validar su ejecución en lenguaje ensamblador mediante simulación.

2. Desarrollo

En esta práctica se partió de un procesador que ya contaba con el ciclo de búsqueda y con las instrucciones `LDAB` y `LDAA` en modo inmediato, `ABA` en modo inherente y `JMP` en modo extendido. A partir de ese punto se agregaron seis rutinas de microprograma: `LDAB` (INM) con el código `D6`, `CMP` (INH), `JE` (EXT), `STAA` (EXT), `LDAA` (EXT) y `LDAB` (EXT). Con ellas se escribieron, en lenguaje ensamblador, tres programas que se cargaron en la memoria RAM mediante el archivo `mem_content.mif`.

2.1. Arquitectura del procesador

La Figura 1 muestra la arquitectura utilizada. Los datos se procesan en dos acumuladores de 8 bits (A y B) y en una unidad de procesamiento aritmético (UPA), cuyas banderas se almacenan en el registro de códigos de condición (CCR). Los registros de 16 bits (índices X y Y, registro auxiliar RA, apuntador de pila y contador de programa PC) se conectan a los buses internos C (parte alta), D (parte baja) y E (bus de direcciones). La memoria externa se accede a través del registro de direcciones, que se carga con la señal $\overline{\text{CBD}}$.

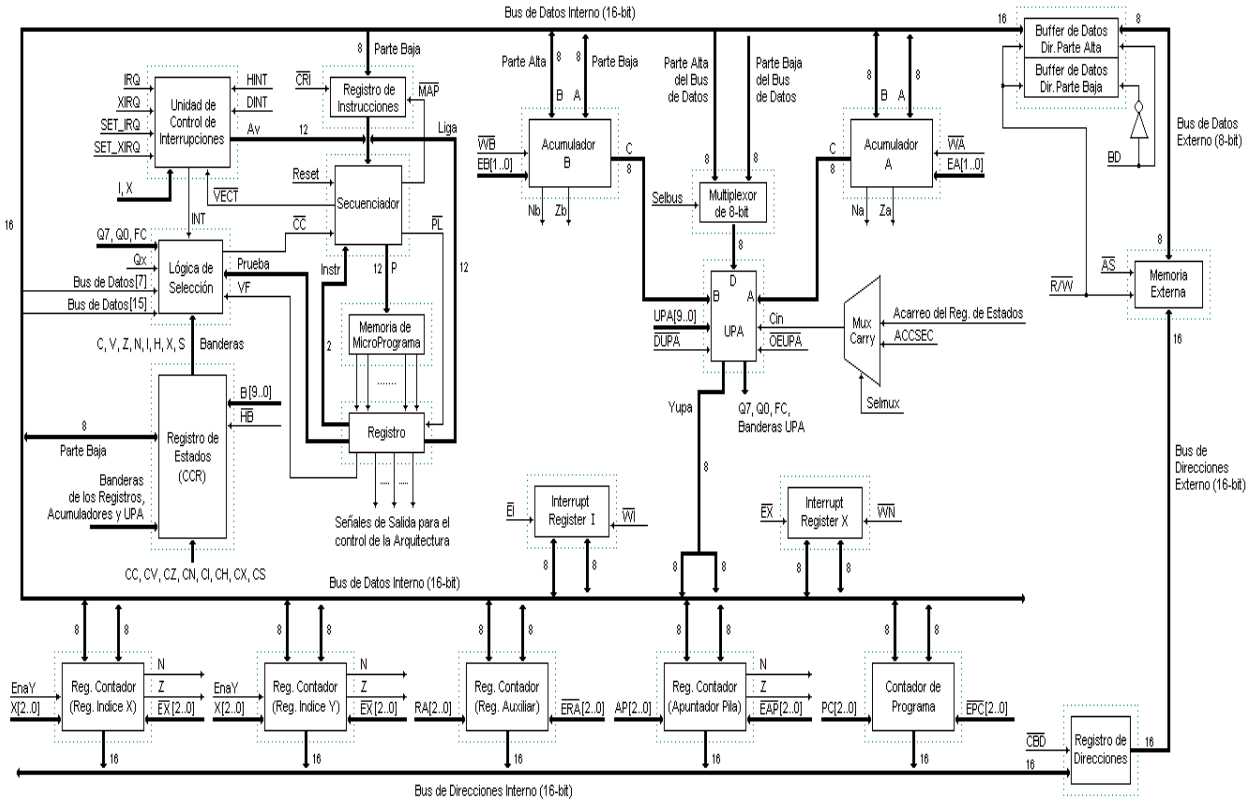


Figura 1: Arquitectura del procesador CISC 68HC11 empleada en la práctica.

La unidad de control es un secuenciador microprogramado. El registro de estado presente (`Edo_Pres`) direcciona la memoria de microprograma (`memory.vhd`); la palabra leída se captura en el registro de microinstrucción (`registro_sec`) en cada flanco ascendente del reloj, por lo que las señales de control asociadas a un estado se ejecutan durante el ciclo siguiente (mientras que el registro de direcciones y la RAM actúan en el flanco descendente). La lógica de selección compara la entrada elegida por el campo de prueba con el bit VF mediante una compuerta XNOR:

$$CC = \overline{\text{Prueba} \oplus VF},$$

y el bloque `logica_seleccion.vhd` interpreta los dos bits del campo de instrucción, según la Tabla 1.

Tabla 1: Códigos de instrucción del secuenciador.

$I_1 I_0$	Nombre	Siguiente estado
`00`	Continuar	$N+1$.
`01`	Salto condicional	Liga si $CC = 1$; $N+1$ si $CC = 0$.
`10`	MAP (decodificación)	Dirección formada por el opcode concatenado con `0`.
`11`	Vector	Rutina de interrupción si $CC = 1$; $N+1$ si $CC = 0$.

Para obtener un salto incondicional se selecciona la entrada `AUX(GND)` (`11000`), que siempre vale cero, con $VF = 0$; de esta forma $CC = 1$ y siempre se carga la liga. Las entradas usadas en esta práctica son `INT` (`01111`), `Z` (`10010`) y `AUX(GND)` (`11000`), tomadas de la hoja `entradas` del archivo de Excel proporcionado por el profesor.

2.2. Formato y dimensionamiento de la memoria de microprograma

Cada palabra de la memoria de microprograma tiene la siguiente organización:

$$\underbrace{P_4 \ldots P_0}_{\text{prueba}}\ \underbrace{VF}_{\text{valor}}\ \underbrace{I_1\, I_0}_{\text{instrucción}}\ \underbrace{L_{11} \ldots L_0}_{\text{liga}}\ \underbrace{\overline{\text{CRI}} \ldots \text{ACCSEC}}_{\text{salidas de control}}.$$

Tabla 2: Campos de la palabra de microprograma.

Campo	Bits	Longitud	Función
Prueba	97 : 93	5	Selecciona la entrada que se evalúa (bandera, `INT`, `GND`, etc.).
VF	92	1	Valor con el que se compara la entrada seleccionada.
Instrucción	91 : 90	2	Modo de secuenciamiento (Tabla 1).
Liga	89 : 78	12	Microdirección de salto.
Salidas	77 : 0	78	Acumuladores, UPA, registros, memoria, interrupciones y banderas.

El número de bits por palabra es

$$m = 5 + 1 + 2 + 12 + 78 = 98 \text{ bits},$$

y como la dirección del secuenciador tiene 12 bits, el espacio de la memoria es

$$2^{12} \times 98 = 4096 \times 98 = 401\,408 \text{ bits}.$$

En `memory.vhd` sólo se describen las microdirecciones utilizadas; cualquier otra dirección entrega la palabra por omisión (todas las salidas inactivas).

2.3. Ciclo de búsqueda y decodificación

La Figura 2 muestra la carta ASM del ciclo de búsqueda y de las instrucciones en modo inmediato e inherente. En `008H` el PC se copia al registro de direcciones; en `009H` se lee el opcode, se guarda en el registro de instrucciones y se incrementa el PC; en `00AH` la instrucción MAP salta a la dirección opcode $\|$ `0`. Por ejemplo, el opcode `D6` conduce a `D60H` y `1C` a `1C0H`. Al terminar, cada rutina regresa a `009H` después de cargar nuevamente el registro de direcciones con el PC.

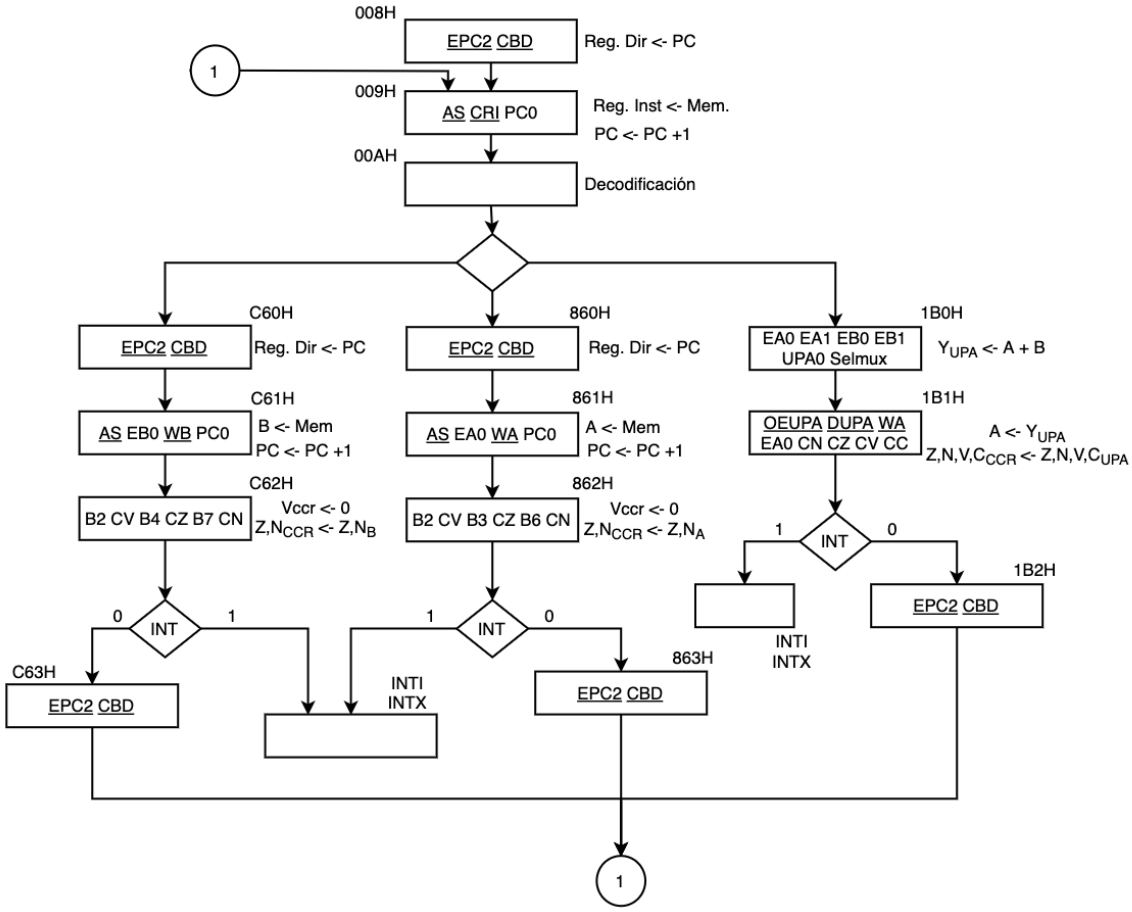


Figura 2: Carta ASM del ciclo de búsqueda y de las instrucciones `LDAB` (INM), `LDAA` (INM) y `ABA` (INH).

La Figura 3 muestra la carta ASM del modo extendido para `JMP` y `STAA`. En ambos casos los dos bytes que siguen al opcode se leen en el registro auxiliar: primero la parte alta (con $BD = 1$) y después la parte baja; en cada lectura se incrementa el PC. Con la dirección completa en RA, `JMP` la copia al PC y `STAA` la usa como dirección de escritura. Cabe señalar que, aunque el diagrama de referencia anota $Y_{\text{UPA}} \leftarrow A \oplus 0$ en `7E0H`, las señales activas (`nEPC2`, `nCBD`) ejecutan la transferencia Reg. Dir $\leftarrow$ PC, tal como se implementa en `memory.vhd`.

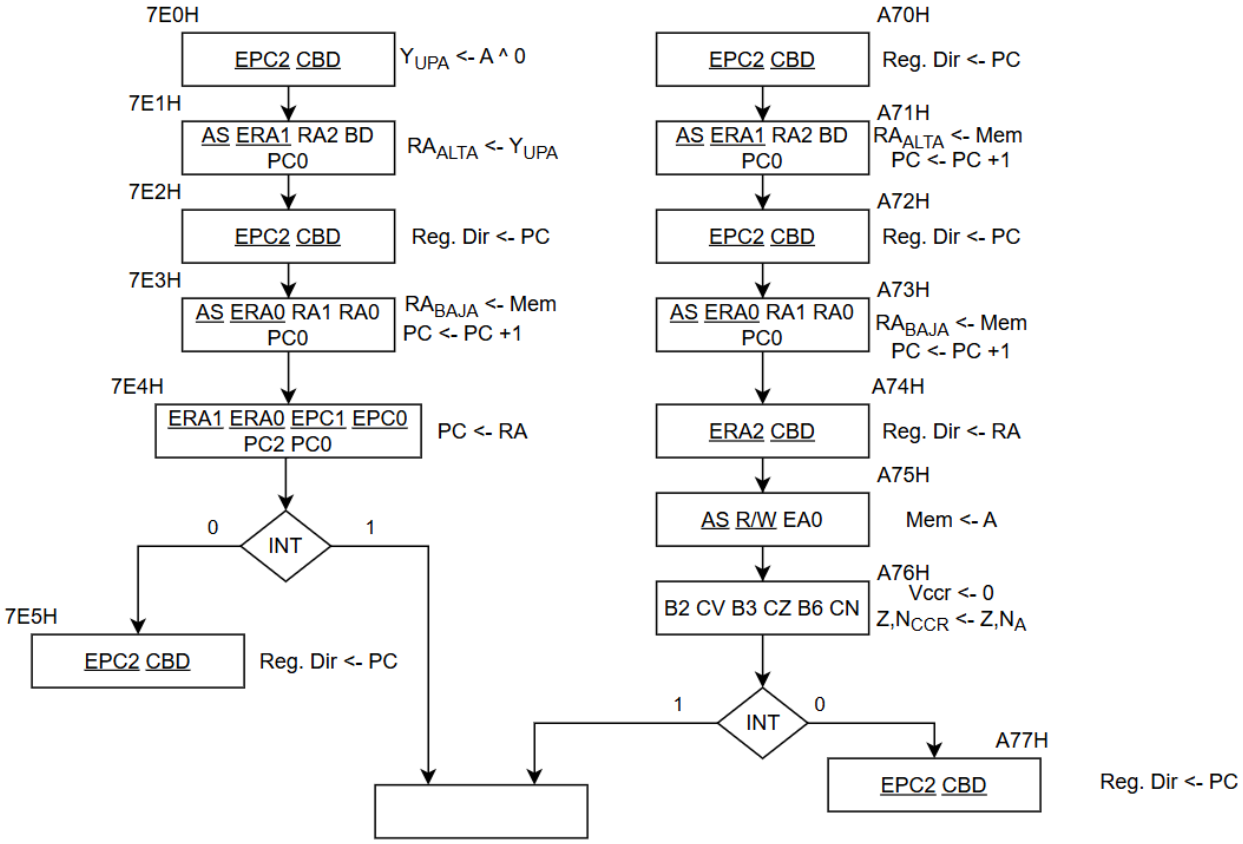


Figura 3: Carta ASM de las instrucciones `JMP` (EXT) y `STAA` (EXT).

2.4. Procedimiento para microprogramar las instrucciones

Las palabras de control se construyeron en la hoja de cálculo proporcionada por el profesor (`salidas_68HC11_CISC-practica7.xlsx`). En la hoja `salidas`, cada fila corresponde a una microdirección y cada columna a una señal; la columna `CW` concatena los 98 valores de la fila. El procedimiento seguido para cada instrucción nueva fue:

1. Identificar en la carta ASM las transferencias de cada estado (por ejemplo, $\text{RA}_{\text{BAJA}} \leftarrow \text{Mem}$, $\text{PC} \leftarrow \text{PC} + 1$).
2. Copiar las filas de una instrucción ya implementada cuya secuencia fuera parecida.
3. Cambiar únicamente las salidas necesarias (habilitaciones, escrituras, función de la UPA, banderas) y, en su caso, los campos de prueba, VF, instrucción y liga.
4. Copiar la cadena `CW` de cada fila al archivo `memory.vhd`, en una condición `elsif` con la microdirección correspondiente.

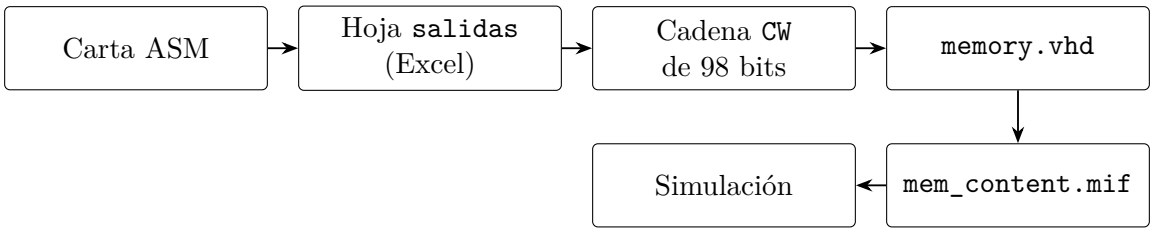


Figura 4: Flujo de trabajo seguido para agregar cada instrucción.

Las Tablas 3 a 7 muestran, para cada estado, la transferencia realizada y las señales que difieren de la palabra por omisión. Las señales con prefijo `n` son activas en bajo. La columna *Sec.* indica los campos de secuenciamiento como `prueba/VF/instrucción/liga`; un guion indica `00000/0/00/000` (continuar a $N+1$).

2.4.1. LDAB (INM), opcode D6

La carga inmediata de B ya existía en las microdirecciones `C60`–`C63`. Como en los programas se utilizó el opcode `D6`, se copiaron las cuatro filas a `D60`–`D63` sin modificar ninguna salida. El estado `D62` actualiza las banderas con los valores de B ($B7$ y $B4$ seleccionan N_B y Z_B) y $B2$ limpia V.

Tabla 3: Microprograma de `LDAB` (INM) en `D60`–`D63`.

Dir.	Transferencia	Señales activas	Sec.
`D60`	Reg. Dir ← PC	nEPC2=0, nCBD=0	–
`D61`	$B \leftarrow$ Mem, PC ← PC+1	EB0=1, nWB=0, PC0=1, nAS=0	–
`D62`	$V \leftarrow 0$; $Z, N \leftarrow Z_B, N_B$	B7, B4, B2, CN, CV, CZ	`01111/1/11/000`
`D63`	Reg. Dir ← PC	nEPC2=0, nCBD=0	`11000/0/01/009`

2.4.2. CMP (INH), opcode 1C

`CMP` realiza $A - B$ para actualizar las banderas, pero no guarda el resultado. Se tomó como referencia `ABA` (`1B0`–`1B2`), porque ambas conectan los dos acumuladores a la UPA. Los cambios fueron:

- En `1C0` se agregó `UPA5=1`, que cambia la función de la UPA de suma a resta $R - S - \overline{C_{in}}$ con $R = A$ y $S = B$, y `ACCSEC=1`, que selecciona el acarreo personalizado para que el resultado sea $A - B$.
- En `1C1` se eliminaron `EA0=1` y `nWA=0`, de modo que la salida de la UPA no se escribe en A. También se quitó `CH`, porque la comparación sólo actualiza C, N, V y Z.
- `1C2` es igual a `1B2`.

Tabla 4: Microprograma de `CMP` (INH) en `1C0`–`1C2`.

Dir.	Transferencia	Señales activas	Sec.
`1C0`	$Y_{UPA} \leftarrow A - B$	EB1, EB0, EA1, EA0, **UPA5**, UPA0, selmux, **ACCSEC**	–
`1C1`	$C, N, V, Z \leftarrow C, N, V, Z_{UPA}$	nOEUPA=0, nDUPA=0, CC, CN, CV, CZ	`01111/1/11/000`
`1C2`	Reg. Dir $\leftarrow$ PC	nEPC2=0, nCBD=0	`11000/0/01/009`

2.4.3. JE (EXT), opcode `7D`

`JE` salta a la dirección de 16 bits que sigue al opcode cuando los operandos comparados son iguales ($Z = 1$). Se copió la rutina de `JMP` (`7E0`–`7E5`) y se modificó sólo el campo de secuenciamiento de `7D3`: se seleccionó la prueba `10010` (Z), con $VF = 0$, instrucción `01` y liga `7D5`. La palabra completa de `7D3` queda así:

$$\underbrace{10010}_{Z}\ \underbrace{0}_{VF}\ \underbrace{01}_{\text{salto}}\ \underbrace{0111\,1101\,0101}_{\texttt{7D5}}\ \underbrace{1001001000\ldots0010}_{\text{salidas iguales a } \texttt{7E3}}.$$

Si $Z = 0$, la entrada coincide con VF, $CC = 1$ y el secuenciador carga la liga `7D5`, omitiendo `7D4`; el PC conserva la dirección siguiente al `JE`. Si $Z = 1$, $CC = 0$ y se continúa en `7D4`, donde se ejecuta PC $\leftarrow$ RA. Las salidas de `7D3` no cambian, por lo que el byte bajo de la dirección se lee en ambos casos y el PC siempre avanza tres bytes. La Figura 5 muestra las cartas ASM resultantes para `CMP` y `JE`.

Tabla 5: Microprograma de `JE` (EXT) en `7D0`–`7D5`.

Dir.	Transferencia	Señales activas	Sec.
`7D0`	Reg. Dir $\leftarrow$ PC	nEPC2=0, nCBD=0	–
`7D1`	$\text{RA}_{\text{ALTA}} \leftarrow$ Mem, PC $\leftarrow$ PC+1	nERA1=0, RA2, PC0, nAS=0, BD	–
`7D2`	Reg. Dir $\leftarrow$ PC	nEPC2=0, nCBD=0	–
`7D3`	$\text{RA}_{\text{BAJA}} \leftarrow$ Mem, PC $\leftarrow$ PC+1	nERA0=0, RA1, RA0, PC0, nAS=0	`10010/0/01/7D5`
`7D4`	PC $\leftarrow$ RA	nERA1=0, nERA0=0, nEPC1=0, nEPC0=0, PC2, PC0	`01111/1/11/000`
`7D5`	Reg. Dir $\leftarrow$ PC	nEPC2=0, nCBD=0	`11000/0/01/009`

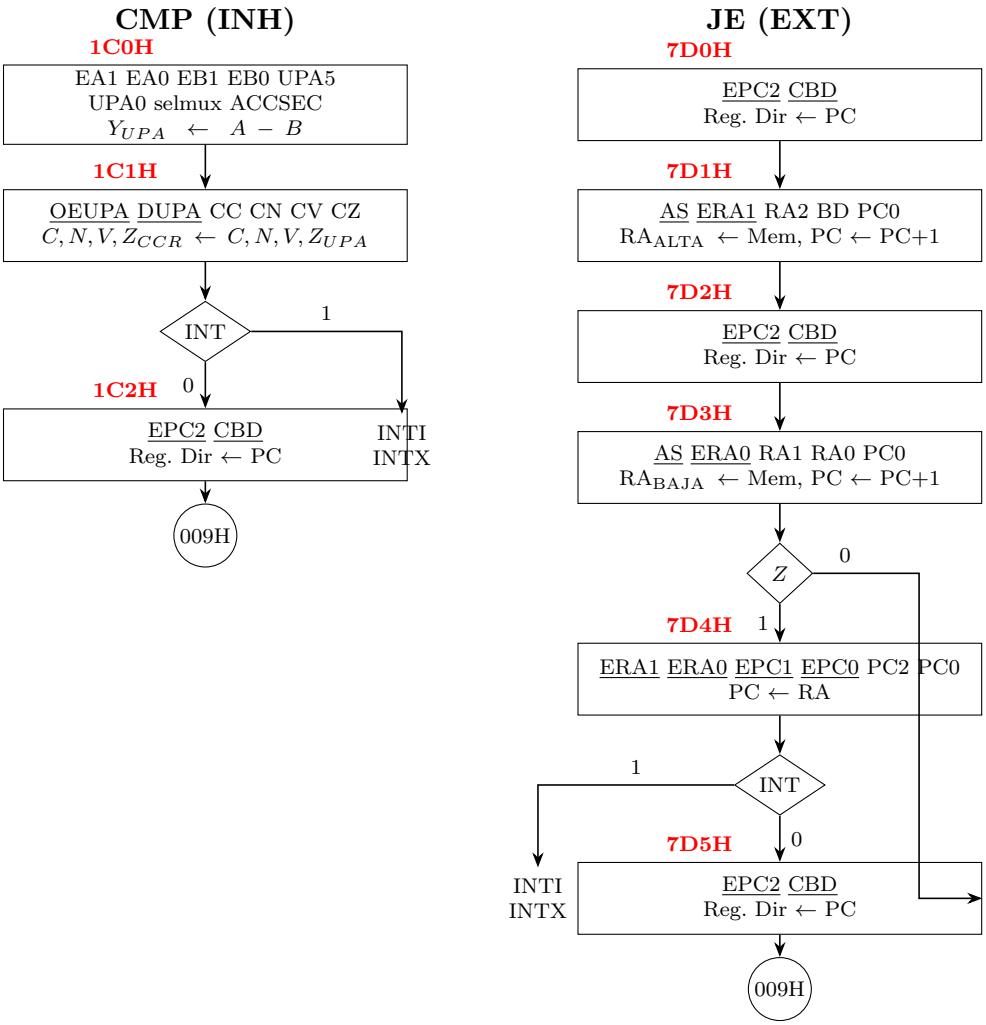


Figura 5: Cartas ASM de las instrucciones `CMP` (INH) y `JE` (EXT). Las señales subrayadas son activas en bajo.

2.4.4. STAA (EXT), opcode `A7`

`STAA` se implementó directamente a partir de la carta ASM de la Figura 3. Los estados `A70`–`A73` son iguales a `7E0`–`7E3`, porque la obtención de la dirección es la misma que en `JMP`. En lugar de copiar RA al PC, en `A74` se carga el registro de direcciones con RA ($\overline{\text{ERA2}}$, $\overline{\text{CBD}}$) y en `A75` se escribe A en la memoria ($\overline{\text{AS}}$, $\overline{\text{R/W}}$, EA0). El estado `A76` actualiza Z y N con el valor de A y limpia V; para ello se copió la fila `862`. Finalmente, `A77` es igual a `863`.

Tabla 6: Microprograma de STAA (EXT) en A70–A77.

Dir.	Transferencia	Señales activas	Sec.
A70–A73	Lectura de la dirección en RA	Iguales a 7E0–7E3	–
A74	Reg. Dir ← RA	nERA2=0, nCBD=0	–
A75	Mem ← A	EA0=1, nAS=0, nRW=0	–
A76	$V \leftarrow 0$; $Z, N \leftarrow Z_A, N_A$	B6, B3, B2, CN, CV, CZ	01111/1/11/000
A77	Reg. Dir ← PC	nEPC2=0, nCBD=0	11000/0/01/009

2.4.5. LDAA (EXT), opcode 96

La carga extendida de A tiene la misma secuencia que STAA, salvo que la memoria se lee en lugar de escribirse. Por ello se copió la rutina A70–A77 y sólo se modificó el sexto estado: en 965 se regresó nRW a 1 (lectura) y se activó nWA=0, manteniendo EA0=1 y nAS=0. Así se obtiene la transferencia $A \leftarrow$ Mem.

2.4.6. LDAB (EXT), opcode F6

Para la carga extendida de B se copió la rutina de LDAA (EXT) y en F65 se cambiaron las señales del acumulador: se desactivaron EA0 y nWA y se activaron EB0=1 y nWB=0, para obtener $B \leftarrow$ Mem. El estado F66 se conservó igual a 966, por lo que evalúa Z y N con el contenido de A (bits $B6$ y $B3$) y no con el de B. El efecto de esta decisión se observa en la simulación del ejercicio 3.

Tabla 7: Diferencias entre STAA, LDAA y LDAB en modo extendido. Los demás estados son iguales.

Instrucción	Sexto estado	Transferencia	Señales activas
STAA (EXT)	A75	Mem ← A	EA0=1, nAS=0, **nRW=0**
LDAA (EXT)	965	$A \leftarrow$ Mem	EA0=1, nAS=0, **nWA=0**
LDAB (EXT)	F65	$B \leftarrow$ Mem	**EB0=1**, nAS=0, **nWB=0**

2.4.7. Resumen de las instrucciones agregadas

Tabla 8: Instrucciones agregadas en la práctica.

Instrucción	Opcode	Bytes	Microdirecciones	Referencia utilizada
LDAB (INM)	D6	2	D60–D63	Copia de C60–C63.
CMP (INH)	1C	1	1C0–1C2	ABA, con resta y sin escritura en A.
JE (EXT)	7D	3	7D0–7D5	JMP, con prueba de Z en 7D3.
STAA (EXT)	A7	3	A70–A77	Carta ASM de modo extendido.
LDAA (EXT)	96	3	960–967	STAA, con lectura en 965.
LDAB (EXT)	F6	3	F60–F67	LDAA (EXT), con escritura en B en F65.

Los opcodes corresponden al mapa de instrucciones definido para la práctica y no necesariamente a los del 68HC11 comercial.

2.5. Módulos implementados en VHDL

A diferencia de las prácticas anteriores, la arquitectura base del procesador ya está descrita en el proyecto, por lo que únicamente se modificó la memoria de microprograma dentro del secuenciador.

Memoria de microprograma (`memory.vhd`). Almacena las palabras de control de 98 bits correspondientes a cada microdirección del secuenciador. El Código 1 muestra las rutinas agregadas en `memory.vhd`; cada cadena binaria proviene de la columna `CW` en la hoja `salidas`.

Código 1: Microinstrucciones agregadas en `memory.vhd`.

```
    --LDAB (INM)
    elsif(dir= X"D60") then data <= "00000000000000000000100100100000000000011011100001110001110000110000111111000000000000000000000010";
    elsif(dir= X"D61") then data <= "00000000000000000000101000100000000000011011100001110001110001111110100000000000000000000000000010";
    elsif(dir= X"D62") then data <= "01111111000000000000100100100000000000011011100001110001110001111111000000010010100011100000010";
    elsif(dir= X"D63") then data <= "11000001000000001001100100100000000000011011100001110001110000110000111111000000000000000000000010";

    --CMP (INH)
    elsif(dir= X"1C0") then data <= "00000000000000000000111111100000100001111111000011100011100011100011111110000000000000000000011";
    elsif(dir= X"1C1") then data <= "01111111000000000000100100100000000000001110000111000111000111000111111110000000000000001111000010";
    elsif(dir= X"1C2") then data <= "11000001000000001001100100100000000000011011100001110001110000110000111111000000000000000000000010";

    --JE (EXT)
    elsif(dir= X"7D0") then data <= "00000000000000000000100100100000000000011011100001110001110000110000111111000000000000000000000010";
    elsif(dir= X"7D1") then data <= "00000000000000000000100100100000000000011011100001010011100011100111111010000000000000000000000010";
    elsif(dir= X"7D2") then data <= "00000000000000000000100100100000000000011011100001110001110000110000111111000000000000000000000010";
    elsif(dir= X"7D3") then data <= "10010001011111010101100100100000000000011011100001100111110001110011111110100000000000000000000010";
    elsif(dir= X"7D4") then data <= "01111111000000000000100100100000000000011011100001000011100010010111111110000000000000000000000010";
    elsif(dir= X"7D5") then data <= "11000001000000001001100100100000000000011011100001110001110000110000111111000000000000000000000010";

    --STA (EXT)
    elsif(dir= X"A70") then data <= "00000000000000000000100100100000000000011011100001110001110000110000111111000000000000000000000010";
    elsif(dir= X"A71") then data <= "00000000000000000000100100100000000000011011100001010011100011100111111010000000000000000000000010";
    elsif(dir= X"A72") then data <= "00000000000000000000100100100000000000011011100001110001110000110000111111000000000000000000000010";
    elsif(dir= X"A73") then data <= "00000000000000000000100100100000000000011011100001100111110001110011111010000000000000000000000010";
    elsif(dir= X"A74") then data <= "00000000000000000000100100100000000000011011100000110001110000110000111111000000000000000000000010";
    elsif(dir= X"A75") then data <= "00000000000000000000100101100000000000011011100001110001110001110001111100000000000000000000000010";
    elsif(dir= X"A76") then data <= "01111111000000000000100100100000000000011011100001110001110001110001111111000000010011000111000010";
    elsif(dir= X"A77") then data <= "11000001000000001001100100100000000000011011100001110001110000110000111111000000000000000000000010";

    -- LDAA (EXT)
    elsif(dir= X"960") then data <= "00000000000000000000100100100000000000011011100001110001110000110000111111000000000000000000000010";
    elsif(dir= X"961") then data <= "00000000000000000000100100100000000000011011100001010011100011100111111010000000000000000000000010";
    elsif(dir= X"962") then data <= "00000000000000000000100100100000000000011011100001110001110000110000111111000000000000000000000010";
    elsif(dir= X"963") then data <= "00000000000000000000100100100000000000011011100001100111110001110011111010000000000000000000000010";
    elsif(dir= X"964") then data <= "00000000000000000000100100100000000000011011100000110001110000110000111111000000000000000000000010";
    elsif(dir= X"965") then data <= "00000000000000000000100101000000000000011011100001110001110001110001111010000000000000000000000010";
    elsif(dir= X"966") then data <= "01111111000000000000100100100000000000011011100001110001110001110001111111000000010011000111000010";
    elsif(dir= X"967") then data <= "11000001000000001001100100100000000000011011100001110001110000110000111111000000000000000000000010";

    -- LDAB (EXT)
    elsif(dir= X"F60") then data <= "00000000000000000000100100100000000000011011100001110001110000110000111111000000000000000000000010";
    elsif(dir= X"F61") then data <= "00000000000000000000100100100000000000011011100001010011100011100111111010000000000000000000000010";
    elsif(dir= X"F62") then data <= "00000000000000000000100100100000000000011011100001110001110000110000111111000000000000000000000010";
    elsif(dir= X"F63") then data <= "00000000000000000000100100100000000000011011100001100111110001110011111010000000000000000000000010";
    elsif(dir= X"F64") then data <= "00000000000000000000100100100000000000011011100000110001110000110000111111000000000000000000000010";
    elsif(dir= X"F65") then data <= "00000000000000000000101000100000000000011011100001110001110001110001111010000000000000000000000010";
    elsif(dir= X"F66") then data <= "01111111000000000000100100100000000000011011100001110001110001110001111111000000010011000111000010";
    elsif(dir= X"F67") then data <= "11000001000000001001100100100000000000011011100001110001110000110000111111000000000000000000000010";
```

2.6. Programas en ensamblador

Los programas se escribieron en `mem_content.mif`, con `WIDTH=8`, `DEPTH=256`, direcciones en decimal (`ADDRESS_RADIX=UNS`) y datos en hexadecimal (`DATA_RADIX=HEX`). En los listados de ensamblador las direcciones se expresan en hexadecimal.

2.6.1. Ejercicio 1: suma repetitiva

```
B = 0x02;
A = 0x00;
while (1) {
    A = A + B;
}
```

Código 2: Ejercicio 1 en ensamblador.

```
Dir.   Bytes      Instruccion
0000   D6 02      LDAB #$02      ; B <- 02
0002   86 00      LDAA #$00      ; A <- 00
0004   1B         ABA            ; A <- A + B
0005   7E 00 04   JMP  $0004     ; regresar a la suma
```

Código 3: Contenido de `mem_content.mif` para el ejercicio 1.

```
CONTENT BEGIN
    0  : D6;   -- LDAB inm
    1  : 02;
    2  : 86;   -- LDAA inm
    3  : 00;
    4  : 1B;   -- ABA inh
    5  : 7E;   -- JMP ext
    6  : 00;
    7  : 04;   -- 04: ciclo de suma; 08: salto a LDAA #FF
    8  : 86;   -- LDAA inm
    9  : FF;
    [10..255] : 00;
END;
```

Además del ciclo infinito, se realizó una segunda prueba cambiando el byte bajo de la dirección del salto a `08`. En ese caso `JMP` conduce a la instrucción `LDAA #$FF` ubicada en `0008`, lo que permite verificar que el PC toma la dirección indicada por los operandos.

2.6.2. Ejercicio 2: comparación de acumuladores

```
B = 0x05;
A = valor;          // 0x05 (iguales) o 0x09 (diferentes)
if (A != B) {
    A = 0x48;
}
A = 0x63;
```

La condición $A \neq B$ se implementa con `CMP` y `JE`: si los acumuladores son iguales, `JE` salta directamente a la asignación final; si son diferentes, el programa continúa con la asignación de `48`.

Código 4: Ejercicio 2 en ensamblador.

```
Dir.   Bytes      Instruccion
0000   D6 05      LDAB #$05      ; B <- 05
0002   86 09      LDAA #$09      ; A <- 09 (05 para la prueba de igualdad)
0004   1C         CMP            ; A - B, actualiza Z
0005   7D 00 0A   JE   $000A     ; si A = B, saltar a FIN
0008   86 48      LDAA #$48      ; A <- 48 (solo si A != B)
000A   86 63      LDAA #$63      ; FIN: A <- 63
```

Código 5: Contenido de `mem_content.mif` para el ejercicio 2.

```
CONTENT BEGIN
    0  : D6;   -- LDAB inm
    1  : 05;
    2  : 86;   -- LDAA inm
    3  : 09;   -- 05 para la prueba de igualdad
    4  : 1C;   -- CMP inh (A - B)
    5  : 7D;   -- JE ext
    6  : 00;
    7  : 0A;
    8  : 86;   -- LDAA inm
    9  : 48;
    10 : 86;   -- LDAA inm
    11 : 63;
    [12..255] : 00;
END;
```

2.6.3. Ejercicio 3: comparación de variables en memoria

En este ejercicio las variables `temp` y `res` se encuentran en la RAM, en las direcciones `0017` y `0018` (posiciones 23 y 24 del MIF). Por ello se emplean las cargas y el almacenamiento en modo extendido.

```
temp = 0x0A;
res  = valor;       // 0x0A (iguales) o 0x1B (diferentes)
if (res != temp) {
    res = 0x48;
}
res = 0x63;
while (1) { }
```

Código 6: Ejercicio 3 en ensamblador.

```
Dir.   Bytes      Instruccion
0000   F6 00 17   LDAB $0017     ; B <- temp
0003   96 00 18   LDAA $0018     ; A <- res
0006   1C         CMP            ; A - B, actualiza Z
0007   7D 00 0F   JE   $000F     ; si res = temp, saltar a FIN
000A   86 48      LDAA #$48      ; A <- 48
000C   A7 00 18   STAA $0018     ; res <- 48
000F   86 63      LDAA #$63      ; FIN: A <- 63
0011   A7 00 18   STAA $0018     ; res <- 63
0014   7E 00 14   JMP  $0014     ; ciclo final
0017   0A                       ; temp
0018   1B                       ; res (0A para la prueba de igualdad)
```

Código 7: Contenido de `mem_content.mif` para el ejercicio 3.

```
CONTENT BEGIN
    0  : F6;   -- LDAB ext
    1  : 00;
    2  : 17;   -- temp (0x17 -> 23)
    3  : 96;   -- LDAA ext
    4  : 00;
    5  : 18;   -- res (0x18 -> 24)
    6  : 1C;   -- CMP inh (A - B)
    7  : 7D;   -- JE ext
    8  : 00;
```

```
    9  : 0F;
    10 : 86;    -- LDAA inm
    11 : 48;
    12 : A7;    -- STAA ext
    13 : 00;
    14 : 18;
    15 : 86;    -- LDAA inm
    16 : 63;
    17 : A7;    -- STAA ext
    18 : 00;
    19 : 18;
    20 : 7E;    -- JMP ext a 0x14
    21 : 00;
    22 : 14;
    23 : 0A;    -- temp
    24 : 1B;    -- res (0A para la prueba de igualdad)
    [25..255] : 00;
END;
```

3. Simulación

Las simulaciones funcionales se realizaron en Quartus con un reloj de 10 ns de periodo, `RESET=1` e `IRQn=XIRQn=1`, es decir, sin solicitudes de interrupción. Se observan los acumuladores (`ACCA_D`, `ACCB_D`), el registro auxiliar (`AUX_D`), el contador de programa (`PC_D`), la habilitación de la RAM (`csnn`, activa en bajo), la bandera Z y el estado presente del secuenciador (`Edo_Pres`).

Cada estado presente dura un ciclo de reloj (10 ns); debido al registro de microinstrucción (`registro_sec`), las señales de control generadas en un estado se reflejan en la ruta de datos durante el ciclo siguiente. Como los valores de `Edo_Pres` no caben completos en la ventana de formas de onda, Quartus sólo muestra el carácter central de cada microdirección: `0` corresponde a `000`, `008`, `009` o `00A`; `6` a `D6x`, `86x`, `96x` o `F6x`; `B` a `1Bx`; `C` a `1Cx`; `D` a `7Dx`; `E` a `7Ex`, y `7` a `A7x`. Contando los estados de cada grupo se identifica con exactitud la instrucción ejecutada y la rama tomada en el microprograma.

3.1. Ejercicio 1: suma repetitiva

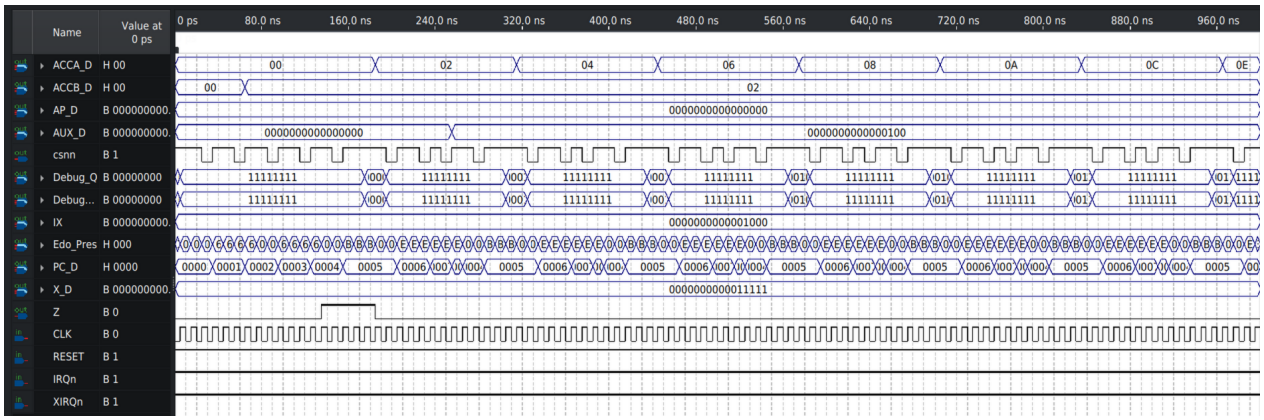


Figura 6: Simulación del ejercicio 1 con salto a `0004`.

En la Figura 6, el secuenciador inicia en `000`, cuya liga lleva a `008`; después recorre `009` y `00A` (cuatro estados `0` iniciales), y el PC pasa de `0000` a `0001` en 25 ns. La instrucción MAP lleva el opcode `D6` a `D60`. Durante `D60`–`D63` (primer grupo de cuatro estados `6`, de 35 a 75 ns) se lee el operando y B cambia a `02` en 65 ns. El segundo grupo de cuatro estados `6` (de 95 a 135 ns) corresponde a `860`–`863`: A se carga con `00` y, como el resultado es cero, Z cambia a 1 en 135 ns tras ejecutarse `862`.

A continuación aparecen tres estados `B` (`1B0`–`1B2`, de 165 a 195 ns). Al concluir `1B1`, la salida de la UPA se escribe en A, que cambia a `02` en 185 ns, y Z regresa a 0. Después siguen seis estados `E` (`7E0`–`7E5`, de 215 a 275 ns): en `7E1` y `7E3` se leen los bytes `00` y `04`, por lo que `AUX_D` toma el valor `0004` en 255 ns; en `7E4` se ordena PC ← RA y el PC adopta `0004` en 265 ns (tras mostrar transitoriamente `0008` debido al incremento previo de `7E3`).

A partir de ese punto la secuencia de estados presentes se repite: `009`, `00A`, `1B0`–`1B2`, `009`, `00A`, `7E0`–`7E5`. Son 13 estados, es decir, 130 ns por iteración, que coinciden exactamente con el intervalo entre cambios sucesivos de A (`02` en 185 ns, `04` en 315 ns, `06` en 445 ns, …, `0E` en 965 ns). En el PC se repite el patrón `0005`, `0006`, `0007`, `0008`, `0004`, y B permanece en `02`.

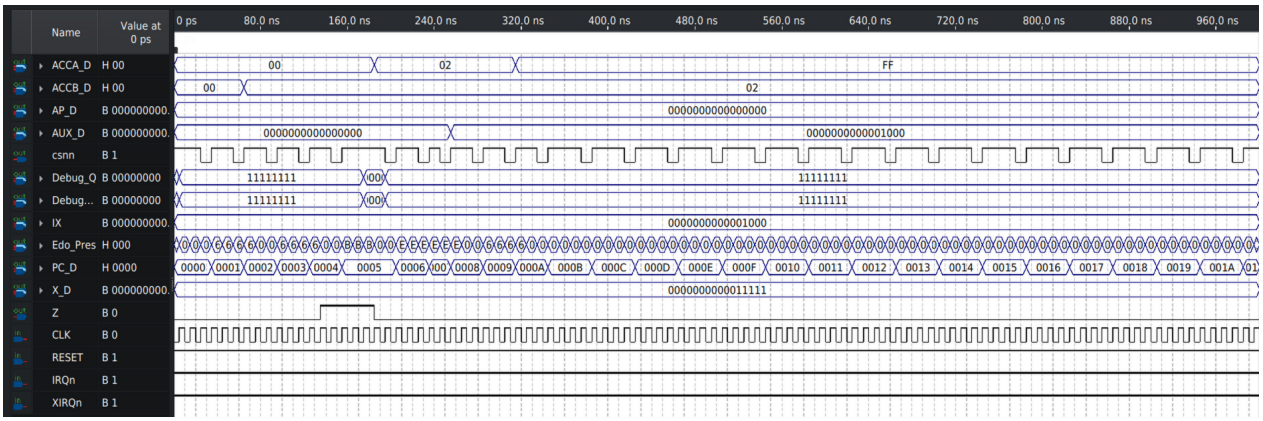


Figura 7: Simulación del ejercicio 1 con salto a `0008`.

En la Figura 7 la secuencia es idéntica hasta el primer `JMP`, pero ahora `AUX_D` toma `0008` en 255 ns y el PC salta a esa dirección en 265 ns. Después del salto aparece un grupo de cuatro estados `6` (`860`–`863`, de 295 a 335 ns) y A cambia de `02` a `FF` en 315 ns. Desde `000A` la memoria contiene `00`; la instrucción MAP lleva ese opcode a la microdirección `000`, cuya liga regresa a `008`. Por ello `Edo_Pres` sólo muestra estados `0` (`000`, `008`, `009`, `00A`) y el PC aumenta en uno cada cuatro ciclos (40 ns), sin modificar los acumuladores.

3.2. Ejercicio 2: comparación de acumuladores

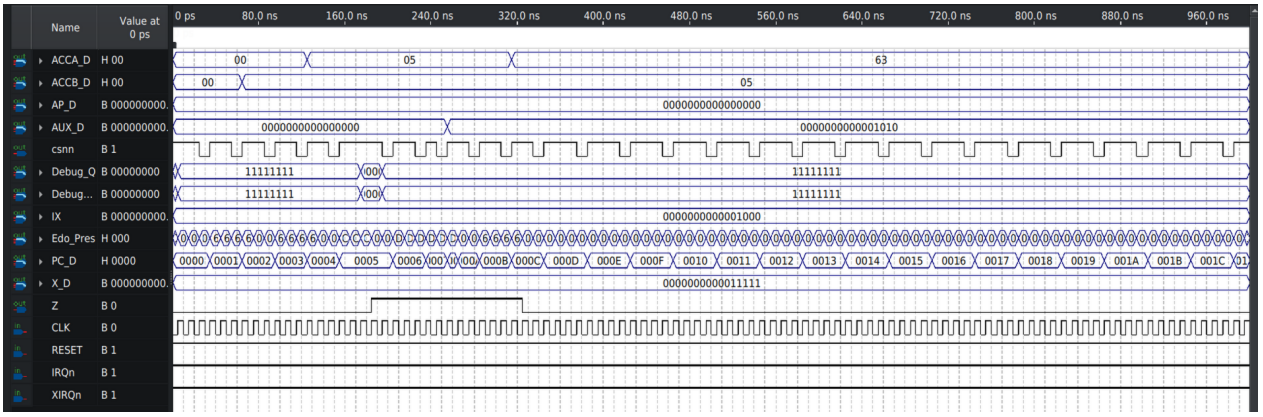


Figura 8: Simulación del ejercicio 2 con $A = B =$ `05`.

En la Figura 8 los dos primeros grupos de estados `6` cargan $B =$ `05` (65 ns) y $A =$ `05` (125 ns). Después aparecen tres estados `C` (`1C0`–`1C2`, de 155 a 185 ns); en `1C1` se actualizan las banderas con $A - B = 0$ y Z cambia a 1 en 175 ns, mientras A conserva `05`.

El grupo siguiente tiene seis estados `D`: `7D0`–`7D5` (de 205 a 265 ns). En `7D3` se prueba Z; como $Z = 1$ y $VF = 0$, $CC = 0$ y el secuenciador continúa en `7D4`, donde el PC se carga con

AUX_D= 000A en 255 ns. En PC_D se observa la secuencia 0006, 0007, 0008, 000A: la dirección 0008 aparece sólo durante un ciclo (transitoriamente) por el incremento efectuado en 7D3, pero 7D4 la sobrescribe de inmediato con 000A, de modo que la instrucción en 0008 nunca se busca ni se ejecuta. Finalmente se ejecuta 860–863 (de 285 a 325 ns), A cambia directamente de 05 a 63 en 315 ns y Z regresa a 0 en 325 ns porque el dato cargado es distinto de cero.

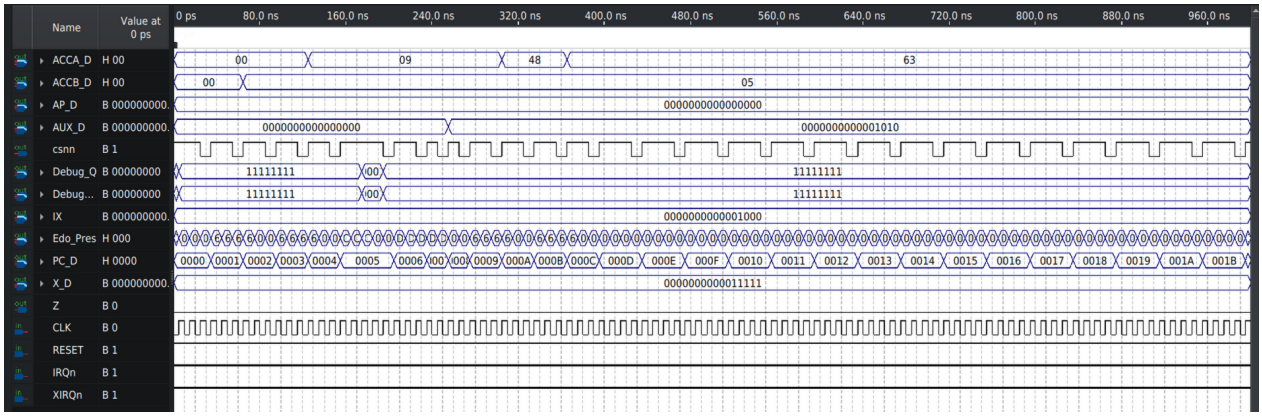


Figura 9: Simulación del ejercicio 2 con $A =$ 09 y $B =$ 05.

En la Figura 9 se carga $A =$ 09 en 125 ns. La comparación produce $A - B =$ 04, por lo que Z permanece en 0 durante toda la simulación. En este caso el grupo de estados D tiene sólo cinco estados (de 205 a 255 ns): en 7D3, $Z = 0$ coincide con $VF = 0$, $CC = 1$ y se carga la liga 7D5, omitiendo 7D4. Aunque AUX_D también recibe 000A en 255 ns, esa dirección no se copia al PC, que continúa secuencialmente en 0008 y 0009.

Después del JE se observan dos grupos de cuatro estados 6: el primero carga 48 en 305 ns y el segundo 63 en 365 ns. Al comparar ambas pruebas, el primer cambio de A después del JE ocurre exactamente 10 ns antes en el caso diferente (305 ns frente a 315 ns), que es la duración del estado 7D4 omitido. El valor final de A es 63 en los dos casos; la diferencia entre ellos se identifica por la rama del secuenciador, el recorrido del PC y el valor intermedio 48.

3.3. Ejercicio 3: comparación de variables en memoria

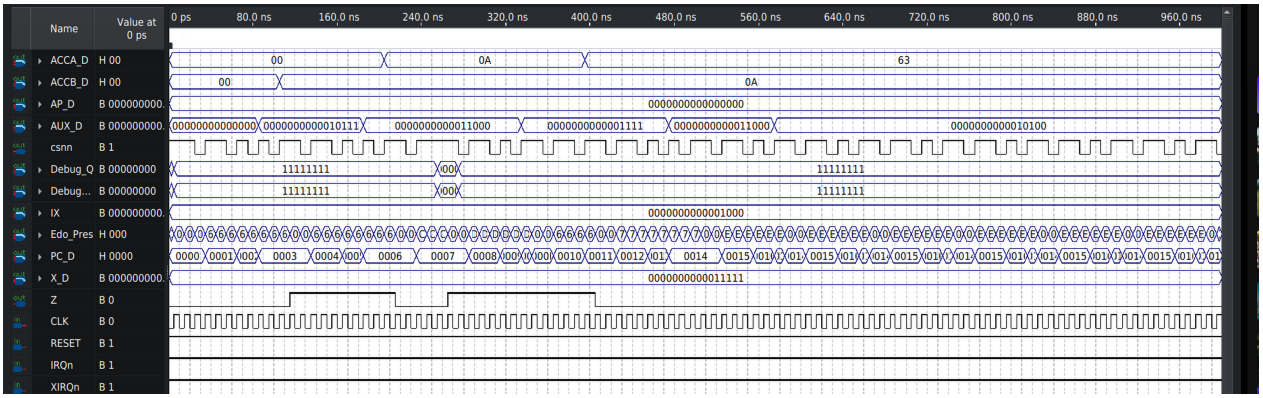


Figura 10: Simulación del ejercicio 3 con `temp`=`res`= `0A`.

En la Figura 10 el primer grupo de ocho estados `6` corresponde a `F60`–`F67` (de 35 a 115 ns). En `F61` y `F63` se leen los bytes de dirección y `AUX_D` toma `0017` en 75 ns; en `F64` el registro de direcciones se carga con RA y en `F65` el dato `0A` se escribe en B (105 ns). En `F66`, Z cambia a 1 en 115 ns aunque $B =$ `0A`: como este estado se copió de `966`, las banderas se calculan con A, que todavía vale `00`. La corrección consiste en usar $B7$ y $B4$ en lugar de $B6$ y $B3$, como en `D62`. Este detalle no afecta al programa, porque Z se vuelve a calcular en `CMP` antes de usarse.

El segundo grupo de ocho estados `6` (`960`–`967`, de 135 a 215 ns) lee la dirección `0018` (`AUX_D`= `0018` en 175 ns) y carga $A =$ `0A` en 205 ns; Z regresa a 0 en 215 ns. En los tres estados `C` (de 235 a 265 ns) la comparación da cero y Z cambia a 1 en 255 ns. Por ello el grupo de estados `D` tiene seis estados (de 285 a 345 ns): se ejecuta `7D4` y el PC salta a `000F` en 335 ns (tras el valor transitorio `0010` de `7D3`), a la vez que `AUX_D` contiene `000F`. Se ejecuta `LDAA #$63` ($A =$ `63` en 395 ns, $Z = 0$ en 405 ns) y luego un solo grupo de ocho estados `7` (`A70`–`A77`, de 425 a 505 ns), en el que `AUX_D` vuelve a `0018` (465 ns) y en `A75` se habilita la escritura (`csnn=0` alrededor de 480–490 ns) para guardar `63` en `res`.

Finalmente aparece el ciclo de `JMP`: `AUX_D` toma `0014` en 565 ns y `Edo_Pres` repite `7E0`–`7E5`, `009`, `00A` (ocho estados, 80 ns por ciclo), mientras el PC recorre `0015`, `0016`, `0017`, `0014`. La simulación no muestra directamente el contenido interno de la RAM; la escritura se verifica por la ejecución de `A75`, el pulso de `csnn`, la dirección en RA y el valor de A.

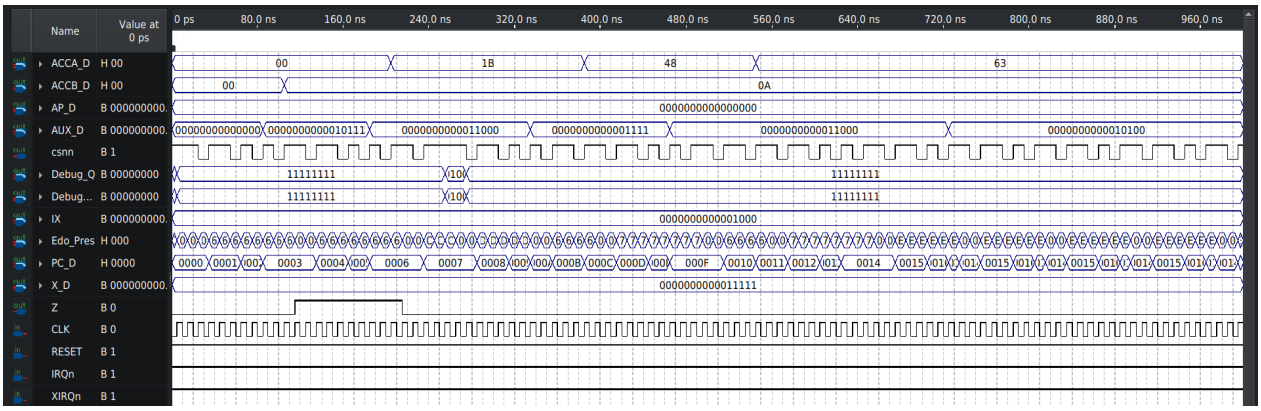


Figura 11: Simulación del ejercicio 3 con `temp`= `0A` y `res`= `1B`.

En la Figura 11 las cargas extendidas producen $B =$ `0A` (105 ns) y $A =$ `1B` (205 ns). La bandera Z se activa en `F66` (115 ns) por la misma razón descrita en la prueba anterior y regresa a 0 al cargar $A =$ `1B` (215 ns); después de `CMP` se mantiene en 0 porque `1B` − `0A` $\neq 0$. El grupo de estados `D` tiene cinco estados (de 285 a 335 ns): en `7D3` se carga la liga `7D5` y el PC continúa en `000A`, aunque `AUX_D` contiene `000F`.

A diferencia del caso anterior, se observan dos pares de rutinas `LDAA`/`STAA`: un grupo de cuatro estados `6` carga $A =$ `48` en 385 ns, un grupo de ocho estados `7` lo guarda en `0018` (`csnn=0` en 470–480 ns); después otro grupo de cuatro estados `6` carga $A =$ `63` en 545 ns y un segundo grupo de estados `7` lo escribe en la misma dirección (`csnn=0` en 630–640 ns). Por ello `AUX_D` permanece en `0018` durante ambos almacenamientos. Al final, el programa entra al mismo ciclo de `JMP` a `0014`.

Tabla 9: Resumen de las simulaciones.

Prueba	Secuencia en A	Comportamiento del secuenciador
1: salto a `0004`	`00, 02, 04, 06, ...`	Ciclo de 13 estados: `009`, `00A`, `1B0`–`1B2`, `009`, `00A`, `7E0`–`7E5`.
1: salto a `0008`	`00, 02, FF`	Salto a `860`–`863`; después, opcodes `00` que recorren `000`–`00A`.
2: iguales	`00, 05, 63`	$Z = 1$; `JE` recorre seis estados y ejecuta `7D4`.
2: diferentes	`00, 09, 48, 63`	$Z = 0$; `JE` recorre cinco estados y omite `7D4`.
3: iguales	`00, 0A, 63`	Un solo `STAA` (`A70`–`A77`) y ciclo final en `0014`.
3: diferentes	`00, 1B, 48, 63`	Dos `STAA` sobre `0018` y ciclo final en `0014`.

4. Conclusión

Percival Ulises Espinoza Matamoros. La práctica me permitió entender cómo se construye una instrucción CISC a partir de microinstrucciones. Al agregar cada instrucción en la hoja de cálculo, tomando como base una rutina ya existente y cambiando sólo las señales necesarias, fue claro que muchas instrucciones comparten la mayor parte de su secuencia; por ejemplo, `JE` sólo requirió modificar los campos de secuenciamiento de un estado de `JMP`. El análisis del estado presente en las simulaciones permitió comprobar que cada rutina recorre las microdirecciones esperadas.

Oscar Iván Montiel Juárez. En esta práctica se diferenció claramente entre modificar el programa en la RAM y modificar el microprograma. Con `mem_content.mif` se escribieron los programas en ensamblador, mientras que en `memory.vhd` se definió qué hace internamente cada instrucción. Las simulaciones mostraron que el número de estados de cada rutina determina el tiempo de ejecución; en el ejercicio 1 cada iteración de la suma requiere 13 ciclos de reloj, y en `JE` la condición falsa ahorra un ciclo al omitir `7D4`.

Amy Valentina Veraza García. La implementación de `CMP` y `JE` ayudó a comprender cómo se construye una decisión en el procesador: `CMP` sólo actualiza las banderas y `JE` utiliza Z mediante la prueba, el bit VF y la liga del secuenciador. Con el modo extendido fue posible trabajar con variables almacenadas en memoria. También se observó que copiar una rutina sin revisar todas sus salidas puede producir efectos no deseados, como la actualización de Z con el acumulador A en `LDAB` (EXT).